

大数据Spark入门专栏-v1.0

大数据Spark入门专栏-v1.0

1. Spark基本概述

- 产生背景
- 什么是Spark?
- 基本特征
- 生态圈

2. Spark环境搭建

- 环境部署
- 集群启停

3. 语言环境搭建

- Java环境
- Scala环境
- Python环境

4. Spark Shell交互式编程

- 使用 `spark-shell` 交互式环境
 - 基本使用
 - 参数
 - 使用方法
 - 代码示例

- 使用 `pyspark` 交互式环境

5. Spark 作业编写与提交指南

- IDE代码开发方法
 - Java代码开发
 - Scala代码开发
 - Python代码开发

- 提交作业

- 提交流程
 - 使用 `spark-submit`
 - Scala 和 Java 提交流程
 - Python 提交流程

- 注意事项

6. Spark RDD创建指南

- RDD创建方法

- 1. 并行化集合 (Parallel Collections)
 - 2. 外部数据源

- 常用外部数据源创建RDD方法

- `textFile`
 - `wholeTextFiles`
 - `sequenceFile`
 - `hadoopRDD`
 - `objectFile`

- Spark RDD Transform算子

- `map(func)`
 - `filter(func)`
 - `flatMap(func)`
 - `mapPartitions(func)`
 - `mapPartitionsWithIndex(func)`
 - `sample(withReplacement, fraction, seed)`
 - `union(otherDataset)`
 - `intersection(otherDataset)`
 - `distinct([numTasks])`

```
groupBy(func)
reduceBy(func)
aggregateByKey(zeroValue)(seqOp, combOp, [numTasks])
sortByKey([ascending], [numTasks])
join(otherDataset, [numTasks])
cogroup(otherDataset, [numTasks])
cartesian(otherDataset)
pipe(command, [envVars])
coalesce(numPartitions)
repartition(numPartitions)
repartitionAndSortWithinPartitions(partitioner)
```

实战案例

清洗WordCount数据

读取name.txt中的姓名并进行去重，然后降序排序

6. Spark RDD Action算子

```
reduce(func)
collect()
count()
first()
take(n)
takeSample(withReplacement, num, [seed])
takeOrdered(n, [ordering])
saveAsTextFile(path)
saveAsSequenceFile(path)
saveAsObjectFile(path)
countByKey()
foreach(func)
```

练习题目：

练习1: 对wordcount词频统计结果的RDD进行操作

练习2: 将RDD Action常用方法融合到wordcount案例中进行练习

练习题目代码

Scala 实现

Java 实现

Python 实现

作者与版本更新计划

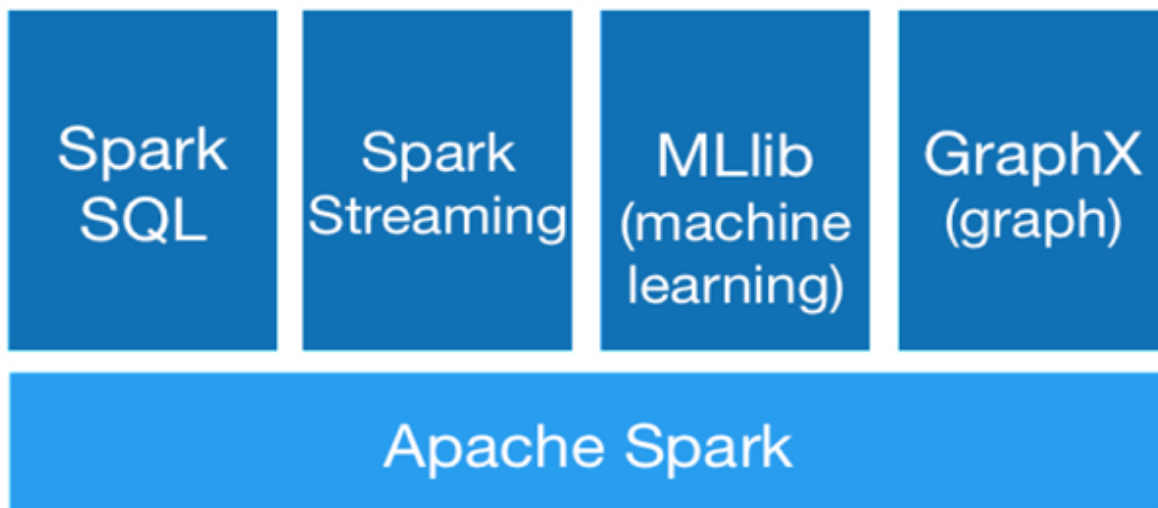
1. Spark基本概述

产生背景

在大数据时代，数据处理的需求日益增长。然而，早期的数据处理框架MapReduce存在一些局限性，如仅支持Map和Reduce两种操作，执行效率较低，主要适用于大规模批处理，而不适合迭代计算、交互式计算和实时流处理等场景。此外，面对众多的计算框架，如批处理的MapReduce、流处理的Storm和Flink、交互式计算的Impala和Presto、机器学习算法的Mahout等，开发者在技术选型上面临困难，学习成本较高。因此，市场迫切需要一个能够统一这些功能的计算框架。

什么是Spark?

Apache Spark是一个由加州大学伯克利分校AMP实验室于2009年开源的大规模分布式通用计算引擎。它在2014年2月成为Apache的顶级项目，并在2017年7月发布了功能更加出色的Spark 2.2.0版本。Spark的设计目标是提供一个快速、易于使用的统一计算框架，以简化技术选型。



Spark由以下几个核心组件构成：

- **Spark Core**：核心计算框架，提供基本的分布式数据操作。
- **Spark SQL**：用于结构化数据查询，支持SQL和DataFrame API。
- **Spark Streaming**：实时流处理组件，能够处理实时数据流。
- **MLlib**：机器学习库，提供常见的机器学习算法。
- **GraphX**：图计算库，用于处理图结构数据。

Spark采用Scala语言开发，并提供了多种运行模式，支持在Standalone集群、Amazon EC2、Hadoop YARN、Mesos以及Kubernetes等多种环境中运行。

基本特征

Spark具有以下显著特征：

- **速度快**：Spark使用先进的DAG调度程序、查询优化器和物理执行引擎，实现了批处理和流处理的高性能运行。
- **易于使用**：Spark提供了80多个高级操作，使得构建并行应用程序变得轻松。它支持使用Scala、Python、R和SQL shell等语言以交互方式使用。
- **通用性**：Spark支持多种库，可以在同一个应用程序中无缝地组合使用，如Spark SQL、MLlib、GraphX和Spark Streaming。
- **多种运行模式**：Spark可以在多种集群管理器上运行，并支持访问HDFS、Alluxio、Apache Cassandra、Apache HBase、Apache Hive等数据源。

生态圈

Spark拥有一个丰富的生态系统，包括各种库和工具，它们共同构成了一个强大的数据处理平台。这个生态系统不仅包括Spark自身的组件，还有许多第三方库和工具，它们扩展了Spark的功能，使其能够更好地适应不同的数据处理需求。

2. Spark环境搭建

环境部署

环境搭建使用脚本安装即可，目前playground脚本支持spark3.5.1、3.4.3、3.2.4、2.4.7版本安装，已经过测试。建议安装with-hadoop版本，例如spark-3.5.1-bin-hadoop3.tgz without-hadoop版本需要额外配置，不然会导致依赖缺失，稍麻烦一些。

这里大家统一安装3.5.1版本的Spark即可。

1、添加安装包。

```
playground add spark-3.5.1-bin-hadoop3.tgz
```

2、配置spark安装信息，根据提示将spark安装在node01-node03上，主节点为node03。

```
playground config spark
```

3、执行脚本安装spark。

```
playground install spark
```

集群启停

1、在node03上进入Spark安装目录（主节点启动命令必须在node03上执行）。

```
cd $SPARK_HOME
```

2、使用脚本启动整个集群，需要注意的是。

```
cd sbin
# 使用一键启动脚本
./start-all.sh

# 也可以依次执行以下脚本
./start-master.sh
./start-worker.sh
```

3、在关闭虚拟机前，需要停止集群。

```
cd $SPARK_HOME/sbin
# 使用一键停止脚本
./stop-all.sh

# 也可以依次执行以下脚本停止
./stop-master.sh
./stop-worker.sh
```

3.语言环境搭建

Java环境

使用脚本安装Hadoop、Spark集群时，默认会安装JDK1.8版本。直接使用即可。

Scala环境

Scala环境需要使用脚本安装，这里选择安装scala-2.12.19版本。

```
playground install scala
```

版本一定要注意，一般Spark 3.x需要2.12.x版本，例如spark3.5.1、3.4.3可以安装scala-2.12.19.tgz，否则作业编译时会有问题。

Python环境

如果使用PySpark的同学，需要安装Python3.8+，目前playground脚本还不支持一键安装，需要手动完成。

4. Spark Shell交互式编程

使用 spark-shell 交互式环境

spark-shell 是 Spark 的交互式命令行环境，它允许用户以交互方式执行 Spark 任务。在 spark-shell 中，已经预创建了一个名为 `sc` 的特殊的 SparkContext。

spark-shell 提供了一个实验和开发 Spark 应用程序的便捷方式，无需编写完整的程序。

基本使用

通过命令行启动 spark-shell，可以执行 Spark 表达式，并即时看到结果。

```
spark-shell
```

参数

spark-shell 支持以下参数来定制会话：

- `--master`: 设置上下文连接到哪个主服务器，local是指在本本地多线程运行，local[4]为4线程。
- `--jars`: 将 JAR 添加到类路径中，多个 JAR 可用逗号分隔。
- `--packages`: 提供以逗号分隔的 Maven 坐标列表，将依赖项（例如 Spark 包）添加到 shell 会话中。
- `--repository`: 将可能存在依赖关系的任何其他存储库（例如 Sonatype）添加到 shell 会话中。

使用方法

以下是一些 spark-shell 的启动方法：

- 在本地模式下运行 spark-shell：

```
./spark-shell --master local[4]
```

- 在本地模式下运行并加载 code.jar：

```
./spark-shell --master local[4] --jars code.jar
```

- 在本地模式下运行并加载 Maven 依赖:

```
./spark-shell --master local[4] --packages "org.example:example:0.1"
```

- 打开 `spark-shell` 帮助, 查看更多信息:

```
./spark-shell --help
```

代码示例

以下是使用 `spark-shell` 完成的词频统计案例:

```
// 启动spark-shell环境
spark-shell --master local[4]

// spark-shell中内置了SparkContext对象sc
val textFile = sc.textFile("file:///root/input.txt")

//将每一行文本分割成单词列表
val words = textFile.flatMap(line => line.split(" "))

// 使用 map 函数将每个单词映射到一个键值对 (单词, 1)
// 然后使用 reduceByKey 函数来合并相同单词的计数:
val counts = words.map(word => (word, 1)).reduceByKey(_ + _)

// 收集词频统计结果, 并根据词频进行降序排序
val result = counts.collect().sortBy(_._2)

// 打印出词频统计的前10个结果
result.take(10).foreach(println)
```

完成操作后, 可以使用 `:quit` 命令退出 Spark Shell。

使用 pyspark 交互式环境

PySpark 是 Apache Spark 的 Python API, 它允许你使用 Python 语言来编写 Spark 应用程序。以下是一个简单的词频统计案例。

需要注意的是, 要先完成Python环境安装, Spark 3.5.1版本推荐安装3.8.x。

1. 启动 PySpark:

打开终端或命令提示符, 启动 PySpark 交互式环境:

```
pyspark
```

2. 读取文本数据:

假设我们有一个文本文件 `input.txt`, 我们想要统计其中的词频。首先, 我们需要将文本数据读入 Spark 的 RDD:

```
textFile = sc.textFile("file:///root/input.txt")
```

3. 分词:

将每一行文本分割成单词列表:

```
words = textFile.flatMap(lambda line: line.split(" "))
```

4. 映射和计数:

使用 `map` 函数将每个单词映射到一个键值对 (单词, 1), 然后使用 `reduceByKey` 函数来合并相同单词的计数:

```
counts = words.map(lambda word: (word, 1)).reduceByKey(lambda a, b: a + b)
```

5. 获取结果并排序:

收集词频统计结果, 并根据词频进行降序排序:

```
result = counts.collect()
sorted_result = sorted(result, key=lambda x: x[1], reverse=True)
```

6. 打印结果:

打印出词频统计的前10个结果:

```
for word, count in sorted_result[:10]:
    print("%s: %i" % (word, count))
```

7. 停止 SparkContext:

完成操作后, 停止 SparkContext:

```
sc.stop()
```

完成操作后, 可以使用 `exit()` 命令退出 Spark Shell。

5. Spark 作业编写与提交指南

Apache Spark 是一个强大的分布式数据处理框架, 提供了多种工具和库来处理大数据。下面将介绍 Spark 的作业编写与提交方法, 包括如何使用 `spark-shell` 交互式环境和如何使用 `spark-submit` 提交作业。

IDE 代码开发方法

Java 代码开发

Maven 依赖:

```
# Spark Core
groupId = org.apache.spark
artifactId = spark-core_2.12
version = 3.5.1
# HDFS
groupId = org.apache.hadoop
artifactId = hadoop-client
version = <your-hdfs-version>
```

代码案例:

```
import java.util.Arrays;
import java.util.List;

import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;

public class SparkBasicExample {
    public static void main(String[] args) {
        // 创建SparkConf配置对象
        SparkConf conf = new
SparkConf().setAppName("SparkBasicExample").setMaster("local[*]");

        // 创建JavaSparkContext
        JavaSparkContext sc = new JavaSparkContext(conf);

        // 创建一个简单的JavaRDD
        JavaRDD<Integer> rdd = sc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6, 7,
8, 9, 10));

        // 应用map转换操作
        JavaRDD<Integer> squares = rdd.map(num -> num * num);

        // 应用filter转换操作
        JavaRDD<Integer> evenSquares = squares.filter(num -> num % 2 == 0);

        // 执行行动操作collect, 收集结果到驱动程序
        List<Integer> evenSquaresCollected = evenSquares.collect();

        // 打印结果
        evenSquaresCollected.forEach(System.out::println);

        // 关闭SparkSession
        sc.stop();
    }
}
```

打包编译需要的插件配置:

```
<build>
  <plugins>
    <!-- 编译源代码 -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version> <!-- 使用适合你项目的版本 -->
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>

    <!-- 复制资源文件 -->
    <plugin>
```



```

        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-resources-plugin</artifactId>
        <version>3.1.0</version> <!-- 使用适合你项目的版本 -->
    </plugin>

    <!-- 运行单元测试 -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>2.22.2</version> <!-- 使用适合你项目的版本 -->
    </plugin>

    <!-- 创建jar包 -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.2.0</version> <!-- 使用适合你项目的版本 -->
        <configuration>
            <!-- 指定主类 -->
            <archive>
                <manifest>
                    <mainClass>org.shuzhou.SparkBaseExample</mainClass>
                </manifest>
            </archive>
        </configuration>
    </plugin>

    <!-- 安装到本地仓库 -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-install-plugin</artifactId>
        <version>2.5.2</version> <!-- 使用适合你项目的版本 -->
    </plugin>

    <!-- 部署到远程仓库 -->
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-deploy-plugin</artifactId>
        <version>2.8.2</version> <!-- 使用适合你项目的版本 -->
    </plugin>
</plugins>
</build>

```

编译：

```
maven clean package
```

Scala代码开发

sbt依赖：

```
scalaVersion := "2.12.19"

libraryDependencies += Seq(
  "org.apache.spark" %% "spark-core" % "3.5.1"
)
```

代码案例:

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkConf

object SparkBasicExample {
  def main(args: Array[String]): Unit = {
    // 创建SparkSession
    val conf = new SparkConf().setAppName("HelloWorld").setMaster("local[2]")
    val sc = new SparkContext(conf)

    // 创建一个简单的RDD
    val rdd = sc.parallelize(1 to 10)

    // 应用map转换操作
    val squares = rdd.map(num => num * num)

    // 应用filter转换操作
    val evenSquares = squares.filter(num => num % 2 == 0)

    // 执行行动操作collect, 收集结果到驱动程序
    val evenSquaresCollected = evenSquares.collect()

    // 打印结果
    evenSquaresCollected.foreach(println)

    sc.stop()
  }
}
```

代码本地运行:

```
sbt run
```

编译为jar包:

```
sbt package
```

会在 `target/scala-2.12/` 目录下生成一个包含所有依赖项的jar文件, 通常命名为 `your-project-name-assembly.jar`。

Python代码开发

可以直接基于 `pyspark` 环境进行开发与测试, 代码提交直接使用 `spark-submit` 即可

```
# 进入pyspark环境
# 需要提前安装Python3.8
pyspark
```

也可以在setup.py中添加以下内容进行依赖安装,

```
install_requires=[
    'pyspark=={site.SPARK_VERSION}'
]
```

或者直接使用pip工具安装pyspark:

```
pip install pyspark
```

示例代码:

```
from pyspark import SparkConf, SparkContext

def main():
    # 创建SparkConf配置对象
    conf = SparkConf().setAppName("App").setMaster("local[*]")

    # 创建SparkContext
    sc = SparkContext(conf=conf)

    # 创建一个简单的RDD
    rdd = sc.parallelize(range(1, 11))

    # 应用map转换操作
    squares = rdd.map(lambda num: num * num)

    # 应用filter转换操作
    even_squares = squares.filter(lambda num: num % 2 == 0)

    # 执行行动操作collect, 收集结果到驱动程序
    even_squares_collected = even_squares.collect()

    # 打印结果
    for num in even_squares_collected:
        print(num)

    # 停止SparkContext
    sc.stop()

if __name__ == "__main__":
    main()
```

提交作业

作业提交是 Spark 的另一种使用方式, 适用于需要运行非交互式应用程序的场景。

提交流程

1. 将应用程序和其所有依赖打包成 JAR 文件。
2. 使用 `spark-submit` 启动应用程序。

使用 `spark-submit`

`spark-submit` 是 Spark 的作业提交脚本，用于将应用程序提交给 Spark 集群执行。以下是 `spark-submit` 的基本用法：

```
./bin/spark-submit \  
  --class org.example.spark.pro.PV \  
  --master local[8] \  
  /root/sparkcore_day1.jar
```

在这个命令中：

- `--class`：指定应用程序的主类。
- `--master`：指定集群的主服务器，`local[8]`代表本地测试运行，并发为8，提交到spark集群则需要指定为 `spark://node03:7070`
- `/root/sparkcore_day1.jar`：指定应用程序的 JAR 文件路径。

Scala 和 Java 提交流程

1. **编写应用程序**: 使用Scala或Java编写Spark应用程序，并确保所有依赖项都已解决。
2. **打包应用程序**: 使用构建工具（如sbt、Maven或Gradle）将应用程序和依赖项打包成JAR文件。
3. **使用spark-submit提交作业**:
使用 `spark-submit` 命令行工具提交作业。指定主类（应用程序的入口类），集群信息，以及其他配置。

示例命令：

```
spark-submit \  
  --class org.shuzhou.SparkBasicExample \  
  --master local[2] \  
  /path/to/your-application.jar
```

参数说明：

- `--class`：指定包含 `main` 方法的Spark应用程序主类。
- `--master`：指定Spark集群的主节点和资源数（例如 `local[8]` 表示本地模式，8个核心）。
- `--deploy-mode`：指定作业部署模式，可以是 `client` 或 `cluster`。`cluster` 模式下，驱动程序在集群中运行。
- `/path/to/your-application.jar`：应用程序JAR文件的路径。

Python 提交流程

1. **编写应用程序**: 使用Python编写Spark应用程序，通常是一个Python脚本。
2. **打包应用程序**: 对于Python应用程序，可以使用 `pyspark` 库，通常不需要打包成JAR文件。但需要确保所有依赖项都可用。
3. **使用spark-submit提交作业**:
使用 `spark-submit` 命令行工具提交Python脚本。指定脚本文件的路径和其他配置。

示例命令：

```
spark-submit \
  --master local[8] \
  --deploy-mode cluster \
  /path/to/your-script.py
```

参数说明:

- `--master`: 指定Spark集群的主节点和资源数。
- `--deploy-mode`: 指定作业部署模式, 对于Python应用程序, 通常使用 `cluster` 模式。
- `/path/to/your-script.py`: Python脚本文件的路径。

注意事项

- 在集群模式下运行作业时, 需要确保所有节点上都安装了Spark, 并且配置了正确的环境变量。
- 对于包含外部依赖的作业, 可以通过 `--jars` 参数指定外部JAR文件, 或者使用 `--packages` 参数来自动下载Maven坐标指定的依赖。
- 提交作业时, 可以根据需要设置其他配置参数, 如内存、超时时间等。

通过遵循上述流程, 无论是Scala、Java还是Python编写的Spark应用程序, 都可以方便地提交到Spark集群上运行。

通过 `spark-shell` 和 `spark-submit`, Spark 提供了灵活的方式来执行数据处理任务。 `spark-shell` 适合于交互式开发和测试, 而 `spark-submit` 适合于运行封装好的应用程序。理解这些基本使用方法对于有效利用 Spark 进行大数据处理至关重要。

6. Spark RDD创建指南

在Apache Spark中, 弹性分布式数据集 (RDD) 是最基本的数据抽象, 它代表了一个不可变的、分布式的数据集合, 可以并行操作。RDD的创建是Spark数据处理的第一步。下面将介绍如何在Spark中创建RDD, 并提供Scala、Java和Python三种语言的代码示例。

RDD创建方法

1. 并行化集合 (Parallel Collections)

并行化集合是通过将程序中的集合转换成RDD。这种方法适合于已经存在于内存中的数据。

Scala 示例:

```
val data = Array(1, 2, 3, 4, 5)
val distData = sc.parallelize(data)
```

Java 示例:

```
Integer[] data = {1, 2, 3, 4, 5};
JavaRDD<Integer> distData = jsc.parallelize(data);
```

Python 示例:

```
data = [1, 2, 3, 4, 5]
distData = sc.parallelize(data)
```

2. 外部数据源

Spark可以从多种外部数据源创建RDD，包括本地文件系统、HDFS、Cassandra、HBase、Amazon S3等。Spark支持多种数据格式，如文本文件、SequenceFiles等。

路径前没有前缀，默认为HDFS，如果是本地数据前缀为file://

Scala 示例:

```
val distFile = sc.textFile("data.txt")
```

Java 示例:

```
JavaRDD<String> distFile = jsc.textFile("data.txt");
```

Python 示例:

```
distFile = sc.textFile("data.txt")
```

常用外部数据源创建RDD方法

以下是一些常用的创建RDD的方法，以及它们在不同语言中的使用示例。

textFile

创建文本类型的RDD。

Scala 示例:

```
val textFileRDD = sc.textFile("path/to/data.txt")
```

Java 示例:

```
JavaRDD<String> textFileRDD = jsc.textFile("path/to/data.txt");
```

Python 示例:

```
textFileRDD = sc.textFile("path/to/data.txt")
```

wholeTextFiles

读取包含多个小文本文件的目录，并返回(文件名，内容)的Pair RDD。

Scala 示例:

```
val wholeTextFilesRDD = sc.wholeTextFiles("path/to/directory")
```

Java 示例:

```
JavaPairRDD<String, String> wholeTextFilesRDD =  
jsc.wholeTextFiles("path/to/directory");
```

Python 示例:

```
from pyspark import SparkContext  
sc = SparkContext("local", "wholeTextFilesExample")  
wholeTextFilesRDD = sc.wholeTextFiles("path/to/directory")
```

sequenceFile

读取SequenceFiles。

Scala 示例:

```
val sequenceFileRDD = sc.sequenceFile[K, V]("path/to/sequencefile")
```

Java 示例:

```
JavaPairRDD<K, V> sequenceFileRDD = jsc.sequenceFile(K.class, V.class,  
"path/to/sequencefile");
```

Python 示例:

```
sequenceFileRDD = sc.sequenceFile("path/to/sequencefile")
```

hadoopRDD

读取Hadoop InputFormat。

Scala 示例:

```
val hadoopRDD = sc.hadoopRDD(aJobConf)
```

Java 示例:

```
JavaRDD<?> hadoopRDD = jsc.hadoopRDD(aJobConf);
```

Python 示例:

```
hadoopRDD = sc.hadoopRDD(aJobConf)
```

objectFile

读取Java对象的序列化文件。

Scala 示例:

```
val objectFileRDD = sc.objectFile[MyClass]("path/to/objectfile")
```

Java 示例:

```
JavaRDD<MyClass> objectFileRDD = jsc.objectFile(MyClass.class,  
"path/to/objectfile");
```

Python 示例:

Python API不支持直接读取Java对象文件，需要使用其他方法来处理。

通过上述示例，我们可以看到，在Spark中创建RDD是一个简单直接的过程，支持多种数据源和语言API。选择合适的创建方法可以有效地将数据加载到Spark中，为后续的数据处理和分析打下基础。

Spark RDD Transform算子

Transform算子是Spark中用于对RDD进行转换操作的函数，它们可以创建新的RDD。以下是根据PPT内容，对每个Transform算子的介绍和三种语言（Scala、Java、Python）的代码示例。

map(func)

对RDD的每个元素应用函数 `func`。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD  
val mappedRDD = rdd.map(x => x * 2) // 将RDD中每个元素乘以2
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个  
包含1到5的RDD  
JavaRDD<Integer> mappedRDD = rdd.map(x -> x * 2); // 将RDD中每个元素乘以2
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD  
mappedRDD = rdd.map(lambda x: x * 2) # 将RDD中每个元素乘以2
```

filter(func)

返回一个新的RDD，包含函数 `func` 返回true的元素。

Scala 示例:

```
val rdd = sc.parallelize(1 to 10) // 创建一个包含1到10的RDD  
val filteredRDD = rdd.filter(x => x % 2 == 0) // 过滤出偶数
```

Java 示例:


```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)); // 创建一个包含1到10的RDD
JavaRDD<Integer> filteredRDD = rdd.filter(x -> x % 2 == 0); // 过滤出偶数
```

Python 示例:

```
rdd = sc.parallelize(range(1, 11)) # 创建一个包含1到10的RDD
filteredRDD = rdd.filter(lambda x: x % 2 == 0) # 过滤出偶数
```

flatMap(func)

类似于map，但每个输入项可以映射到0个或多个输出项。

Scala 示例:

```
val lines = sc.textFile("data.txt") // 读取文本文件并创建RDD
val wordsRDD = lines.flatMap(line => line.split(" ")) // 将每行分割成单词
```

Java 示例:

```
JavaRDD<String> lines = jsc.textFile("data.txt"); // 读取文本文件并创建RDD
JavaRDD<String> wordsRDD = lines.flatMap(line -> Arrays.asList(line.split(" ")).iterator()); // 将每行分割成单词
```

Python 示例:

```
lines = sc.textFile("data.txt") # 读取文本文件并创建RDD
wordsRDD = lines.flatMap(lambda line: line.split(" ")) # 将每行分割成单词
```

mapPartitions(func)

类似于map，但在RDD的每个分区上单独运行。

Scala 示例:

```
val rdd = sc.parallelize(1 to 100) // 创建一个包含1到100的RDD
val mappedPartitionsRDD = rdd.mapPartitions(iter => iter.map(_ + 1)) // 每个分区的元素都加1
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个RDD
JavaRDD<Integer> mappedPartitionsRDD = rdd.mapPartitions(iter -> {
    Iterator<Integer> transformed = new ArrayList<>(iter).stream().map(x -> x + 1).iterator();
    return transformed;
}); // 每个分区的元素都加1
```

Python 示例:

```
rdd = sc.parallelize(range(1, 101)) # 创建一个包含1到100的RDD
mappedPartitionsRDD = rdd.mapPartitions(lambda iter: (x + 1 for x in iter)) #
每个分区的元素都加1
```

mapPartitionsWithIndex(func)

类似于mapPartitions，但也为func提供了表示分区索引的整数值。

Scala 示例:

```
val rdd = sc.parallelize(1 to 100) // 创建一个包含1到100的RDD
val mappedPartitionsWithIndexRDD = rdd.mapPartitionsWithIndex((index, iter) =>
iter.map(x => (index, x))) // 每个分区的元素都与其索引配对
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个
RDD
JavaRDD<Tuple2<Integer, Integer>> mappedPartitionsWithIndexRDD =
rdd.mapPartitionsWithIndex((index, iter) -> {
    Iterator<Tuple2<Integer, Integer>> transformed = new ArrayList<>
(iter).stream().map(x -> new Tuple2<>(index, x)).iterator();
    return transformed;
}); // 每个分区的元素都与其索引配对
```

Python 示例:

```
from pyspark import RDD, Tuple2

def map_partitions_with_index(iter):
    index = 0
    for item in iter:
        yield Tuple2(index, item)
        index += 1

rdd = sc.parallelize(range(1, 101)) # 创建一个包含1到100的RDD
mappedPartitionsWithIndexRDD =
sc.parallelize(rdd).mapPartitionsWithIndex(map_partitions_with_index) # 每个分区
的元素都与其索引配对
```

sample(withReplacement, fraction, seed)

对数据的一小部分进行采样。

Scala 示例:

```
val rdd = sc.parallelize(1 to 100) // 创建一个包含1到100的RDD
val sampledRDD = rdd.sample(false, 0.1, 1234L) // 随机采样10%的元素，不替换
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)); // 创建一个RDD
JavaRDD<Integer> sampledRDD = rdd.sample(false, 0.1, 1234L); // 随机采样10%的元素, 不替换
```

Python 示例:

```
rdd = sc.parallelize(range(1, 101)) # 创建一个包含1到100的RDD
sampledRDD = rdd.sample(False, 0.1, 1234) # 随机采样10%的元素, 不替换
```

union(otherDataset)

返回一个新RDD，包含源数据集和参数中元素的并集。

Scala 示例:

```
val rdd1 = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val rdd2 = sc.parallelize(6 to 10) // 创建一个包含6到10的RDD
val unionRDD = rdd1.union(rdd2) // 创建一个新的RDD，包含rdd1和rdd2的所有元素
```

Java 示例:

```
JavaRDD<Integer> rdd1 = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
JavaRDD<Integer> rdd2 = jsc.parallelize(Arrays.asList(6, 7, 8, 9, 10)); // 创建一个包含6到10的RDD
JavaRDD<Integer> unionRDD = rdd1.union(rdd2); // 创建一个新的RDD，包含rdd1和rdd2的所有元素
```

Python 示例:

```
rdd1 = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
rdd2 = sc.parallelize(range(6, 11)) # 创建一个包含6到10的RDD
unionRDD = rdd1.union(rdd2) # 创建一个新的RDD，包含rdd1和rdd2的所有元素
```

intersection(otherDataset)

返回包含源数据集和参数中元素交集的新RDD。

Scala 示例:

```
val rdd1 = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val rdd2 = sc.parallelize(4 to 8) // 创建一个包含4到8的RDD
val intersectionRDD = rdd1.intersection(rdd2) // 返回交集RDD，即包含4和5
```

Java 示例:

```
JavaRDD<Integer> rdd1 = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
JavaRDD<Integer> rdd2 = jsc.parallelize(Arrays.asList(4, 5, 6, 7, 8)); // 创建一个包含4到8的RDD
JavaRDD intersectionRDD = rdd1.intersection(rdd2); // 返回交集RDD，即包含4和5
```

Python 示例:

```
python rdd1 = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
rdd2 = sc.parallelize(range(4, 9)) # 创建一个包含4到8的RDD
intersectionRDD = rdd1.intersection(rdd2) # 返回交集RDD, 即包含4和5
```

distinct([numTasks])

返回包含源数据集的不同元素的新RDD。

Scala 示例:

```
val rdd = sc.parallelize(Array(1, 2, 2, 3, 3, 3, 4)) // 创建一个包含重复元素的RDD
val distinctRDD = rdd.distinct() // 去重, 返回一个新的RDD, 只包含唯一的元素
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 2, 3, 3, 3, 4)); //
创建一个包含重复元素的RDD
JavaRDD<Integer> distinctRDD = rdd.distinct(); // 去重, 返回一个新的RDD, 只包含唯一的
元素
```

Python 示例:

```
rdd = sc.parallelize([1, 2, 2, 3, 3, 3, 4]) # 创建一个包含重复元素的RDD
distinctRDD = rdd.distinct() # 去重, 返回一个新的RDD, 只包含唯一的元素
```

groupBy(func)

对RDD的元素进行分组。

Scala 示例:

```
val pairsRDD = sc.parallelize(List("a" -> 1, "a" -> 2, "b" -> 3)) // 创建一个键值
对RDD
val groupedRDD = pairsRDD.groupBy(_._1) // 根据键分组
```

Java 示例:

```
JavaPairRDD<String, Integer> pairsRDD = jsc.parallelizePairs("a", 1, "a", 2,
"b", 3); // 创建一个键值对RDD
JavaRDD<Tuple2<String, JavaRDD<Integer>>> groupedRDD = pairsRDD.groupBy(pair ->
pair._1); // 根据键分组
```

Python 示例:

```
pairsRDD = sc.parallelize([("a", 1), ("a", 2), ("b", 3)]) # 创建一个键值对RDD
groupedRDD = pairsRDD.groupBy(lambda pair: pair[0]) # 根据键分组
```

reduceBy(func)

对每个分组的元素进行reduce操作。

Scala 示例:

```
val pairsRDD = sc.parallelize(List("a" -> 1, "a" -> 2, "b" -> 3)) // 创建一个键值对RDD
val reducedRDD = pairsRDD.reduceByKey(_ + _) // 将具有相同键的值相加
```

Java 示例:

```
JavaPairRDD<String, Integer> pairsRDD = jsc.parallelizePairs("a", 1, "a", 2, "b", 3); // 创建一个键值对RDD
JavaPairRDD<String, Integer> reducedRDD = pairsRDD.reduceByKey((a, b) -> a + b);
// 将具有相同键的值相加
```

Python 示例:

```
pairsRDD = sc.parallelize [("a", 1), ("a", 2), ("b", 3)]) # 创建一个键值对RDD
reducedRDD = pairsRDD.reduceByKey(lambda a, b: a + b) # 将具有相同键的值相加
```

aggregateByKey(zeroValue)(seqOp, combOp, [numTasks])

聚合每个键的值。

Scala 示例:

```
val pairsRDD = sc.parallelize(List("a" -> 1, "a" -> 2, "b" -> 3)) // 创建一个键值对RDD
val aggregatedRDD = pairsRDD.aggregateByKey(0)(
  (seq, x) => seq + x, // 序列函数
  (comb1, comb2) => comb1 + comb2 // 组合函数
)
```

Java 示例:

```
JavaPairRDD<String, Integer> pairsRDD = jsc.parallelizePairs("a", 1, "a", 2, "b", 3); // 创建一个键值对RDD
JavaPairRDD<String, Integer> aggregatedRDD = pairsRDD.aggregateByKey(0,
  (seq, x) -> seq + x, // 序列函数
  (comb1, comb2) -> comb1 + comb2 // 组合函数
);
```

Python 示例:

```
pairsRDD = sc.parallelize [("a", 1), ("a", 2), ("b", 3)]) # 创建一个键值对RDD
aggregatedRDD = pairsRDD.aggregateByKey(0)(
  lambda seq, x: seq + x, # 序列函数
  lambda comb1, comb2: comb1 + comb2 # 组合函数
)
```

sortByKey([ascending], [numTasks])

按键排序。

Scala 示例:

```
val pairsRDD = sc.parallelize(List("a" -> 1, "b" -> 2, "c" -> 3)) // 创建一个键值对RDD
val sortedRDD = pairsRDD.sortByKey(false) // 降序排序
```

Java 示例:

```
JavaPairRDD<String, Integer> pairsRDD = jsc.parallelizePairs("a", 1, "b", 2, "c", 3); // 创建一个键值对RDD
JavaPairRDD<String, Integer> sortedRDD = pairsRDD.sortByKey(false); // 降序排序
```

Python 示例:

```
pairsRDD = sc.parallelize([("a", 1), ("b", 2), ("c", 3)]) # 创建一个键值对RDD
sortedRDD = pairsRDD.sortByKey(False) # 降序排序
```

join(otherDataset, [numTasks])

进行join操作。

Scala 示例:

```
val pairsRDD1 = sc.parallelize(List("a" -> 1, "b" -> 2)) // 创建一个键值对RDD
val pairsRDD2 = sc.parallelize(List("a" -> "apple", "b" -> "banana")) // 创建另一个键值对RDD
val joinedRDD = pairsRDD1.join(pairsRDD2) // 内连接
```

Java 示例:

```
JavaPairRDD<String, Integer> pairsRDD1 = jsc.parallelizePairs("a", 1, "b", 2); // 创建一个键值对RDD
JavaPairRDD<String, String> pairsRDD2 = jsc.parallelizePairs("a", "apple", "b", "banana"); // 创建另一个键值对RDD
JavaPairRDD<String, Tuple2<Integer, String>> joinedRDD = pairsRDD1.join(pairsRDD2); // 内连接
```

Python 示例:

```
pairsRDD1 = sc.parallelize([("a", 1), ("b", 2)]) # 创建一个键值对RDD
pairsRDD2 = sc.parallelize([("a", "apple"), ("b", "banana")]) # 创建另一个键值对RDD
joinedRDD = pairsRDD1.join(pairsRDD2) # 内连接
```

cogroup(otherDataset, [numTasks])

对两个RDD中的元素进行分组。

Scala 示例:

```
val pairsRDD1 = sc.parallelize(List("a" -> 1, "b" -> 2)) // 创建一个键值对RDD
val pairsRDD2 = sc.parallelize(List("a" -> "apple", "b" -> "banana")) // 创建另一个键值对RDD
val cgroupedRDD = pairsRDD1.cogroup(pairsRDD2) // 根据键对来自两个RDD的元素进行分组
```

Java 示例:

```
JavaPairRDD<String, Integer> pairsRDD1 = jsc.parallelizePairs("a", 1, "b", 2);  
// 创建一个键值对RDD  
JavaPairRDD<String, String> pairsRDD2 = jsc.parallelizePairs("a", "apple", "b",  
"banana"); // 创建另一个键值对RDD  
JavaPairRDD<String, Tuple2<Iterable<Integer>, Iterable<String>>> cogroupedRDD =  
pairsRDD1.cogroup(pairsRDD2); // 根据键对来自两个RDD的元素进行分组
```

Python 示例:

```
pairsRDD1 = sc.parallelize([("a", 1), ("b", 2)]) # 创建一个键值对RDD  
pairsRDD2 = sc.parallelize([("a", "apple"), ("b", "banana")]) # 创建另一个键值对  
RDD  
cogroupedRDD = pairsRDD1.cogroup(pairsRDD2) # 根据键对来自两个RDD的元素进行分组
```

cartesian(otherDataset)

计算笛卡尔积。

Scala 示例:

```
val rdd1 = sc.parallelize(1 to 2) // 创建一个包含1和2的RDD  
val rdd2 = sc.parallelize(3 to 4) // 创建一个包含3和4的RDD  
val cartesianRDD = rdd1.cartesian(rdd2) // 计算两个RDD的笛卡尔积
```

Java 示例:

```
JavaRDD<Integer> rdd1 = jsc.parallelize(Arrays.asList(1, 2)); // 创建一个包含1和2的  
RDD  
JavaRDD<Integer> rdd2 = jsc.parallelize(Arrays.asList(3, 4)); // 创建一个包含3和4的  
RDD  
JavaRDD<Tuple2<Integer, Integer>> cartesianRDD = rdd1.cartesian(rdd2); // 计算两  
个RDD的笛卡尔积
```

Python 示例:

```
rdd1 = sc.parallelize([1, 2]) # 创建一个包含1和2的RDD  
rdd2 = sc.parallelize([3, 4]) # 创建一个包含3和4的RDD  
cartesianRDD = rdd1.cartesian(rdd2) # 计算两个RDD的笛卡尔积
```

pipe(command, [envVars])

通过shell命令管道RDD的每个分区。

Scala 示例:

```
val rdd = sc.parallelize("hello world" :: "spark is great" :: Nil) // 创建一个RDD  
val pipedRDD = rdd.pipe("grep spark") // 对RDD中的每一行使用grep命令过滤包含'spark'的  
行
```

Java 示例:

```
JavaRDD<String> rdd = jsc.parallelize(Arrays.asList("hello world", "spark is great")); // 创建一个RDD
JavaRDD<String> pipedRDD = rdd.pipe("grep spark"); // 对RDD中的每一行使用grep命令过滤包含'spark'的行
```

Python 示例:

```
rdd = sc.parallelize(["hello world", "spark is great"]) # 创建一个RDD
pipedRDD = rdd.pipe("grep spark") # 对RDD中的每一行使用grep命令过滤包含'spark'的行
```

coalesce(numPartitions)

减少分区数。

Scala 示例:

```
val rdd = sc.parallelize(1 to 100) // 创建一个包含1到100的RDD
val coalescedRDD = rdd.coalesce(10) // 将RDD的分区数减少到10
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10), 100); // 创建一个包含1到100的RDD
JavaRDD<Integer> coalescedRDD = rdd.coalesce(10); // 将RDD的分区数减少到10
```

Python 示例:

```
rdd = sc.parallelize(range(1, 101), 100) # 创建一个包含1到100的RDD
coalescedRDD = rdd.coalesce(10) # 将RDD的分区数减少到10
```

repartition(numPartitions)

重新分区。

Scala 示例:

```
val rdd = sc.parallelize(1 to 100) // 创建一个包含1到100的RDD
val repartitionedRDD = rdd.repartition(10) // 重新分区RDD为10个分区
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10), 5); // 创建一个包含1到100的RDD
JavaRDD<Integer> repartitionedRDD = rdd.repartition(10); // 重新分区RDD为10个分区
```

Python 示例:

```
rdd = sc.parallelize(range(1, 101), 5) # 创建一个包含1到100的RDD
repartitionedRDD = rdd.repartition(10) # 重新分区RDD为10个分区
```

repartitionAndSortWithinPartitions(partitioner)

重新分区并排序。

Scala 示例:

```
val rdd = sc.parallelize(1 to 100) // 创建一个包含1到100的RDD
val repartitionedAndSortedRDD = rdd.repartitionAndSortWithinPartitions(new
HashPartitioner(10)) // 重新分区并排序
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9,
10), 5); // 创建一个包含1到100的RDD
JavaRDD<Integer> repartitionedAndSortedRDD =
rdd.repartitionAndSortWithinPartitions(new HashPartitioner(10)); // 重新分区并排序
```

Python 示例:

```
rdd = sc.parallelize(range(1, 101), 5) # 创建一个包含1到100的RDD
repartitionedAndSortedRDD = rdd.repartitionAndSortWithinPartitions(10) # 重新分区并排序
```

实战案例

清洗WordCount数据

在前面词频统计的基础上，去除单词"word"和"count"。

Scala 示例:

```
val lines = sc.textFile("wordcount_output.txt") // 读取wordcount输出的文本文件
val words = lines.flatMap(_.split(" ")).filter(!Set("word",
"count").contains(_)) // 分割每一行，然后去除"word"和"count"
```

Java 示例:

```
JavaRDD<String> lines = jsc.textFile("wordcount_output.txt"); // 读取wordcount输出的文本文件
JavaRDD<String> words = lines.flatMap(line -> Arrays.asList(line.split("
")).iterator())
                        .filter(word -> !Arrays.asList("word",
"count").contains(word)); // 分割每一行，然后去除"word"和"count"
```

Python 示例:

```
lines = sc.textFile("wordcount_output.txt") # 读取wordcount输出的文本文件
words = lines.flatMap(lambda line: line.split(" ")).filter(lambda word: word not
in ["word", "count"]) # 分割每一行，然后去除"word"和"count"
```

读取name.txt中的姓名并进行去重，然后降序排序

Scala 示例:

```
val names = sc.textFile("name.txt") // 读取包含姓名的文本文件
val namesDistinct = names.distinct() // 去除重复的姓名
val namesReduced = namesDistinct.map(_._1).reduceByKey(_ + _) // 为每个姓名添加计数，并合并相同的姓名计数
val namesSorted = namesReduced.sortBy(_._2, false).keys // 按计数降序排序，然后获取姓名（丢弃计数）
```

Java 示例:

```
JavaRDD<String> names = jsc.textFile("name.txt"); // 读取包含姓名的文本文件
JavaRDD<String> namesDistinct = names.distinct(); // 去除重复的姓名
JavaPairRDD<String, Integer> namesReduced = namesDistinct.mapToPair(word -> new Tuple2<>(word, 1))
                                                                .reduceByKey((a, b) -> a + b);

// 为每个姓名添加计数，并合并相同的姓名计数
JavaRDD<String> namesSorted = namesReduced.sortBy(t -> -t._2, false).keys(); // 按计数降序排序，然后获取姓名（丢弃计数）
```

Python 示例:

```
names = sc.textFile("name.txt") # 读取包含姓名的文本文件
namesDistinct = names.distinct() # 去除重复的姓名
namesReduced = namesDistinct.map(lambda name: (name, 1)).reduceByKey(lambda a, b: a + b) # 为每个姓名添加计数，并合并相同的姓名计数
namesSorted = namesReduced.sortBy(lambda x: -x[1]).keys() # 按计数降序排序，然后获取姓名（丢弃计数）
```

Transform算子是Spark中处理数据的核心，它们提供了丰富的数据处理能力。通过上述示例，我们可以看到如何使用不同的Transform算子来处理和转换数据，以及如何将它们应用到实际问题中。这些示例展示了Spark在不同编程语言中的灵活性和强大能力。

6. Spark RDD Action算子

Action算子是Spark中用于触发实际计算的函数。它们会返回一个值到驱动程序，或者请求整个RDD的计算结果。以下是每个Action算子的介绍、功能描述、以及在Scala、Java和Python中的代码示例和注释。

reduce(func)

使用函数 `func`（接受两个参数并返回一个）来聚合数据集的元素。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val sum: Int = rdd.reduce(_ + _) // 求和
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
int sum = rdd.reduce((a, b) -> a + b).get(); // 求和
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
sum = rdd.reduce(lambda a, b: a + b) # 求和
```

collect()

在驱动程序中将数据集的所有元素作为数组返回。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val collected: Array[Int] = rdd.collect() // 收集所有元素到一个数组
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
JavaRDD<Integer> collected = rdd.collect(); // 收集所有元素到一个数组
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
collected = rdd.collect() # 收集所有元素到一个数组
```

count()

返回数据集中的元素数。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val count: Long = rdd.count() // 计数
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
long count = rdd.count(); // 计数
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
count = rdd.count() # 计数
```

first()

返回数据集的第一个元素。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val first: Int = rdd.first() // 获取第一个元素
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
int first = rdd.first().get(); // 获取第一个元素
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
first = rdd.first() # 获取第一个元素
```

take(n)

返回包含数据集的前 `n` 个元素的数组。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val take: Array[Int] = rdd.take(2) // 取前2个元素
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
JavaRDD<Integer> take = rdd.take(2); // 取前2个元素
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
take = rdd.take(2) # 取前2个元素
```

takeSample(withReplacement, num, [seed])

返回一个数组，其中包含数据集的 `num` 个元素的随机样本，有或没有替换，可选地预先指定随机数生成器种子。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
val sample: Array[Int] = rdd.takeSample(false, 2, 42) // 无放回随机取2个样本
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
JavaRDD<Integer> sample = rdd.takeSample(false, 2, 42); // 无放回随机取2个样本
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
sample = rdd.takeSample(False, 2, 42) # 无放回随机取2个样本
```

takeOrdered(n, [ordering])

使用自然顺序或自定义比较器返回RDD的前 n 个元素。

Scala 示例:

```
val rdd = sc.parallelize(Seq(5, 3, 1, 4, 2)) // 创建一个包含1到5乱序的RDD
val ordered: Array[Int] = rdd.takeOrdered(3)(implicitly[Ordering[Int]].reverse)
// 降序取前3个元素
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(5, 3, 1, 4, 2)); // 创建一个包含1到5乱序的RDD
JavaRDD<Integer> ordered = rdd.takeOrdered(3, new Ordering<Integer>() {
    @Override
    public int compare(Integer a, Integer b) {
        return b.compareTo(a); // 降序
    }
}); // 降序取前3个元素
```

Python 示例:

```
rdd = sc.parallelize([5, 3, 1, 4, 2]) # 创建一个包含1到5乱序的RDD
ordered = rdd.takeOrdered(3, key=lambda x: -x) # 降序取前3个元素
```

saveAsTextFile(path)

将数据集的元素作为文本文件（或文本文件集）写入本地文件系统，HDFS或任何其他Hadoop支持的文件系统的给定目录中。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
rdd.saveAsTextFile("path/to/output") // 保存为文本文件
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
rdd.saveAsTextFile("path/to/output"); // 保存为文本文件
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
rdd.saveAsTextFile("path/to/output") # 保存为文本文件
```

saveAsSequenceFile(path)

(Java and Scala) 将数据集的元素作为Hadoop SequenceFile写入本地文件系统，HDFS或任何其他Hadoop支持的文件系统上的给定路径中。

Scala 示例:

```
val pairRDD = sc.parallelize(1 to 5).map(x => (x, x.toString)) // 创建一个键值对RDD
pairRDD.saveAsSequenceFile("path/to/output") // 保存为SequenceFile
```

Java 示例:

```
JavaPairRDD<Integer, String> pairRDD = jsc.parallelizePairs(
    1, "1", 2, "2", 3, "3", 4, "4", 5, "5"
); // 创建一个键值对RDD
pairRDD.saveAsSequenceFile("path/to/output"); // 保存为SequenceFile
```

Python 不支持直接创建SequenceFile，需要使用Hadoop的Java API或第三方库。

saveAsObjectFile(path)

(Java and Scala) 使用Java序列化以简单格式编写数据集的元素，然后可以使用SparkContext.objectFile()加载。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
rdd.saveAsObjectFile("path/to/output") // 保存为对象文件
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
rdd.saveAsObjectFile("path/to/output"); // 保存为对象文件
```

Python 不支持直接使用saveAsObjectFile。

countByKey()

仅适用于类型 (K, V) 的RDD。返回 (K, Int) 对的散列映射以及每个键的计数。

Scala 示例:

```
val pairRDD = sc.parallelize(List(("a", 1), ("a", 2), ("b", 3))) // 创建一个键值对RDD
val countMap: Map[String, Int] = pairRDD.countByKey() // 计数每个键出现的次数
```

Java 示例:

```
JavaPairRDD<String, Integer> pairRDD = jsc.parallelizePairs(
    "a", 1, "a", 2, "b", 3
); // 创建一个键值对RDD
HashMap<String, Integer> countMap = pairRDD.countByKey(); // 计数每个键出现的次数
```

Python 示例:

```
from pyspark import AccumulatorParam

class CountByKeyParam(AccumulatorParam):
    def zero(self, existingValue):
        return 0

    def addInPlace(self, v1, v2):
        return v1 + v2

    def addAccumulator(self, acc, value):
        acc += value

pairRDD = sc.parallelize [("a", 1), ("a", 2), ("b", 3)] # 创建一个键值对RDD
countMap = pairRDD.countByKey() # 计数每个键出现的次数
```

foreach(func)

在数据集的每个元素上运行函数 func。

Scala 示例:

```
val rdd = sc.parallelize(1 to 5) // 创建一个包含1到5的RDD
rdd.foreach(println) // 打印RDD中的每个元素
```

Java 示例:

```
JavaRDD<Integer> rdd = jsc.parallelize(Arrays.asList(1, 2, 3, 4, 5)); // 创建一个包含1到5的RDD
rdd.foreach(System.out::println); // 打印RDD中的每个元素
```

Python 示例:

```
rdd = sc.parallelize(range(1, 6)) # 创建一个包含1到5的RDD
rdd.foreach(print) # 打印RDD中的每个元素
```

练习题目：

练习1: 对wordcount词频统计结果的RDD进行操作

这个练习的目标是使用不同的Action算子对wordcount统计结果进行操作：

- 返回单词的总数（使用 `count()`）。
- 用两种方法获取RDD的第一条数据（使用 `first()` 和 `take(1)`）。
- 将统计结果保存到本地文件 `result.txt` 中（使用 `saveAsTextFile("path")`）。

练习2: 将RDD Action常用方法融合到wordcount案例中进行练习

请将常用的RDD Action方法应用到wordcount程序中，以加深对Spark RDD操作的理解。

练习题目代码

Scala 实现

练习1:

```
val inputPath = "path/to/input.txt" // 输入文件路径
val outputPath = "path/to/result.txt" // 输出文件路径

val textFile = sc.textFile(inputPath)
val counts = textFile.flatMap(_.split("\\s+")).map(word => (word, 1)).reduceByKey(_ + _)

// 返回单词总数
val totalCount = counts.count()

// 用两种方法拿到RDD的第一条数据
val firstWord = counts.first()
val takeFirstWord = counts.take(1).head

// 将统计结果保存到本地文件result.txt中
counts.saveAsTextFile(outputPath)

println(s"Total count: $totalCount")
println(s"First word: $firstWord")
println(s"Take first word: $takeFirstWord")
```

练习2:

```
// 继续使用上面的代码和变量
// 这里我们演示foreach操作，例如打印每个单词及其计数
counts.foreach(println)
```

Java 实现

练习1:

```
String inputPath = "path/to/input.txt"; // 输入文件路径
String outputPath = "path/to/result.txt"; // 输出文件路径

JavaRDD<String> textFile = jsc.textFile(inputPath);
JavaPairRDD<String, Integer> counts = textFile.flatMap(
    s -> Arrays.asList(s.split("\\s+")).stream().map(word -> new Tuple2<>(word,
1)).collect(Collectors.toList())
).reduceByKey((a, b) -> a + b);

// 返回单词总数
long totalCount = counts.count();

// 用两种方法拿到RDD的第一条数据
String firstWord = counts.first().get()._1;
String takeFirstWord = counts.take(1).get(0)._1;

// 将统计结果保存到本地文件result.txt中
counts.saveAsTextFile(outputPath);

System.out.println("Total count: " + totalCount);
System.out.println("First word: " + firstWord);
System.out.println("Take first word: " + takeFirstWord);
```

练习2:

```
// 继续使用上面的代码和变量
// 这里我们演示foreach操作，例如打印每个单词及其计数
counts.foreach(tuple -> System.out.println(tuple._1 + " : " + tuple._2));
```

Python 实现

练习1:

```
input_path = "path/to/input.txt" # 输入文件路径
output_path = "path/to/result.txt" # 输出文件路径

text_file = sc.textFile(input_path)
counts = (text_file.flatMap(lambda line: line.split(" "))
          .map(lambda word: (word, 1))
          .reduceByKey(lambda a, b: a + b))

# 返回单词总数
total_count = counts.count()

# 用两种方法拿到RDD的第一条数据
first_word = counts.first()
take_first_word = counts.take(1)[0]

# 将统计结果保存到本地文件result.txt中
counts.saveAsTextFile(output_path)

print("Total count:", total_count)
print("First word:", first_word)
print("Take first word:", take_first_word)
```

练习2:

```
# 继续使用上面的代码和变量
# 这里我们演示foreach操作，例如打印每个单词及其计数
def print_word_count(word_count):
    print(word_count)

counts.foreach(print_word_count)
```

Action算子是Spark中执行实际计算并返回结果的关键。它们允许开发者获取整个RDD的处理结果，或者将结果保存到外部存储系统。通过上述示例，我们可以看到如何使用不同的Action算子来处理数据，并执行各种聚合和输出操作。

作者与版本更新计划

关注公众号【数舟】，获取作者最新动态。



目前版本为v1.0，更新时间2024年7月27日。

后续此文档更新与版本发布会同步到知识星球【数舟】中。加入数舟，两倍咖啡的价格获取企业大数据实战经验。

知识整理与创作不易，感谢大家理解与支持！

数舟

数舟

Shu

Zhou

星主：鹏程



📁 231

📖 334

极简IT硬核知识点讲解与分析，提供高品质技术实战文档。每年至少更新100-200篇技术文章。让你几分钟理解什么是湖仓集一体，事务隔离性，WAL两次提交，Raft算法等知识。快速补全认知。技术代码均经过测试运

🌐 知识星球

微信扫码加入星球



鹏程 向你推荐这个有用的星球